beam

User Guide · version 0.1.0

Fast, encrypted file and folder transfer between Linux machines. Use it from the terminal when you want speed and simplicity, or from a live dashboard in your browser when you want to see what is happening.

Parallel TCP streams · AES-256-GCM / ChaCha20-Poly1305 · PAKE pairing codes · Adaptive zstd · Resumable · LAN discovery · CLI + web UI



For Linux · written in C

October 2026

Contents

1 **Introduction**
What beam is, what it does, and the few ideas you need to know

2 **Installation**
Dependencies, building, starting at login, firewall

3 **Quick start**
Pair two machines and send your first folder in five minutes

4 **Pairing and trust**
Pairing codes, one-shot codes, the internet, forgetting a device

5 **Using the terminal**
Sending, receiving, following, cancelling and resuming transfers

6 **Using the web dashboard**
A tour of the browser interface

7 **Getting the most speed**
Benchmarking, streams, compression, cables and disks

8 **Reliability and resuming**
What happens when things go wrong, and how beam recovers

9 **Security**
How devices are authenticated and data is protected

10 **Configuration reference**
Settings, files, ports and environment variables

11 **Troubleshooting and FAQ**
Error messages explained, common questions

12 **Command reference**
Every command and option on one page

13 **For developers**
Project layout, tests, rebuilding this guide

1 Introduction

beam moves files and folders between two Linux computers as fast as the connection between them allows: a single 1 GB video, a photo library, or a source tree with a hundred thousand tiny files.

- **Fast**

  Splits data into 4 MiB chunks and sends them over several TCP connections at once (8 by default). Small files are packed together so thousands of them don't cost thousands of round trips.

- **Private**

  Everything is encrypted and authenticated. Machines prove who they are with keys exchanged once during pairing, so only your devices can send to you.

- **Smart compression**

  Text, logs and databases are compressed with zstd on the fly; videos, archives and ISOs are detected as incompressible and sent as-is at full speed.

- **Resumable**

  If the network drops, a machine crashes or you press Ctrl-C, just run the same command again — only the missing pieces are sent.

- **Zero setup on a LAN**

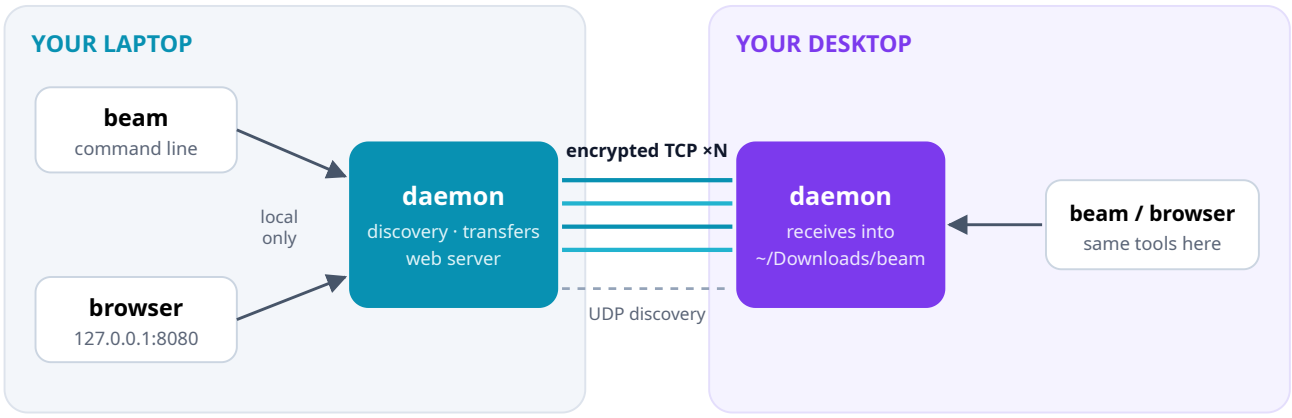
  beam finds other beam machines on your network automatically and shows which network interface and link speed each one is reachable on.

- **Two interfaces**

  A terminal command for speed and scripts, and a browser dashboard with a file picker, live throughput graphs and history.

How it fits together

Each computer runs a small background program, the **daemon**. The daemons talk to each other over the network. You talk to *your* daemon either with the `beam` command or through the web dashboard — both are just remote controls for the same daemon. You never need to start the daemon by hand: any `beam` command starts it if it isn't running.



Words used in this guide

Daemon	The background program on each machine that does the actual work. Started automatically.
Peer	Another computer running beam.
Pairing	A one-time introduction between two of your machines using a short code. Afterwards they recognise each other forever (until you unpair).
One-shot code	A code that lets an *unpaired* machine send you files exactly once — handy for a friend's laptop.
Fingerprint	A 16-character identifier of a machine's key, like `a786fdb2645b25f1`. Shown everywhere so you can tell devices apart.
Transfer	One send operation (any number of files and folders). Each has a small number, its *id*, used by commands like `beam cancel 3`.
Download folder	Where received files go: `~/Downloads/beam` unless you change it.

2 Installation

beam is built from source. It takes about a minute and needs to be done on every machine you want to use.

Requirements

- A Linux distribution (developed on Ubuntu; any modern distribution works).
- GCC 15 or newer, or Clang 19 or newer — the web dashboard is compiled into the program with the C23 `#embed` feature.
- Four small libraries: **libsodium** (encryption), **zstd** (compression), **xxHash** (checksums) and **cJSON**.

Build and install

terminal

```
# 1. Install the build tools and libraries (Debian / Ubuntu)
$ sudo apt install build-essential pkg-config libsodium-dev libzstd-dev libxxhash-dev
libcjson-dev

# 2. Get the code and build it
$ git clone https://github.com/<you>/beam.git
$ cd beam
$ make

# 3. Install into ~/.local/bin (no root needed)
$ make install
installed to /home/you/.local/bin/beam
enable at login with: systemctl --user enable --now beam
```

> **FEDORA / ARCH**
>
> Package names differ slightly: Fedora uses `libsodium-devel libzstd-devel xxhash-devel cjson-devel`; Arch uses `libsodium zstd xxhash cjson`.

Make sure `~/.local/bin` is on your `PATH` (it is by default on Ubuntu after you log in again). Check the install:

terminal

```
$ beam --version
0.1.0
```

Start beam automatically (optional)

Any `beam` command starts the daemon when needed, but a machine can only *receive* while its daemon is running. On a computer that should always be ready to receive — a desktop or home server — let systemd start it when you log in:

terminal

```
$ systemctl --user enable --now beam
# optional: keep it running even when you are logged out
$ sudo loginctl enable-linger $USER
```

Open the firewall

If a firewall is active (Ubuntu ships `ufw`), allow beam's two ports on **every** machine that should receive or be discovered:

terminal

```
$ sudo ufw allow 7878/tcp   # transfers
$ sudo ufw allow 7879/udp   # finding each other on the LAN
```

Uninstall

terminal

```
$ beam stop
$ systemctl --user disable beam; rm ~/.config/systemd/user/beam.service
$ rm ~/.local/bin/beam
$ rm -rf ~/.config/beam      # removes keys, pairings and history
```

3 Quick start

Two machines on the same network: a LAPTOP that sends and a DESKTOP that receives. beam is installed on both (chapter 2).

1 Check that the machines see each other LAPTOP

The first command starts the daemon. After a couple of seconds the other machine appears under "On this network".

laptop

```
$ beam peers
Paired:
  (none — pair with `beam pair`)

On this network:
  desktop              cafb151299672aaa  192.168.1.7:7878  via wlp1s0 (Wi-Fi)
```

Nothing listed? Run any beam command on the desktop too (e.g. `beam status`) so its daemon starts, and check the firewall (chapter 2).

2 Show a pairing code DESKTOP

desktop

```
$ beam pair
Pairing code:  3704-9640-4439

On the other machine run:
  beam pair desktop 3704-9640-4439

(valid for 5 minutes, one attempt)
```

3 Enter the code LAPTOP

laptop

```
$ beam pair desktop 3704-9640-4439
✓ paired with desktop (cafb151299672aaa)
```

The desktop's terminal confirms with `✓ paired with laptop`. You never need to do this again for these two machines.

4 Send something LAPTOP

laptop

```
$ beam send ~/Videos/trip ~/project --to desktop
[████████████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒]  54.2%  610 MiB / 1.1 GiB  8.15 Gbit/s  ETA 0:01
✓ sent to desktop (1.1 GiB, 1502 files) in 1.1s — 8.20 Gbit/s avg
```

The files appear on the desktop in `~/Downloads/beam/trip` and `~/Downloads/beam/project`, with their permissions and modification times preserved.

5 Open the dashboard EITHER

terminal

```
$ beam ui
```

Your browser opens on `http://127.0.0.1:8080` with live graphs, a file picker and history (chapter 6).

BOTH DIRECTIONS

Pairing works both ways. The desktop can now also run `beam send … --to laptop`.

4 Pairing and trust

beam only accepts transfers from machines it knows. There are two ways to become known: pairing (permanent) and one-shot codes (single use).

Pairing two of your machines

Run `beam pair` on one machine to show a code, then `beam pair <name> <code>` on the other, as in the quick start. Rules worth knowing:

- A code is valid for **5 minutes** and allows **one attempt**. A wrong entry uses it up — just run `beam pair` again.
- `<name>` can be the device name shown by `beam peers`, a fingerprint prefix, or an address like `192.168.1.20` or `myserver.example.com:7878`.
- Separators in the code don't matter: `3704 9640 4439` and `370496404439` work too.
- The code never crosses the network. Even someone recording your traffic cannot work it out afterwards (the technique is called a PAKE — see chapter 9).

One-shot codes for machines you don't own

To receive files from a friend's computer without pairing it permanently, ask for a one-shot code on the receiving side:

desktop (receiver)

```
$ beam recv --code
One-shot code:  0271-5949-4950

On the sending machine run:
  beam send <files> --to desktop --code 0271-5949-4950

Waiting for the transfer… (Ctrl-C to stop waiting)
Receiving server.log from laptop
✓ received from laptop (300.0 MiB, 1 files) in 0.3s — 8.33 Gbit/s avg, compressed
  saved to ~/Downloads/beam/server.log
```

friend's laptop (sender)

```
$ beam send server.log --to desktop --code 0271-5949-4950
✓ sent to desktop (300.0 MiB, 1 files) in 0.3s — 8.43 Gbit/s avg, compressed
```

The code is valid for **10 minutes**, for **one** transfer, and leaves no permanent trust behind. In the dashboard, the same thing is the *Receive with code* button.

Sending over the internet

Automatic discovery only works on the local network, but pairing and transfers work anywhere:

1. On the receiver's router, forward **TCP port 7878** to the receiving computer.
2. Pair using the public address: `beam pair home.example.com:7878 <code>` (you can pair on the LAN first, then travel — the pairing stays valid).
3. Send with the address: `beam send big.iso --to home.example.com:7878`. For paired devices, beam also remembers the last address it reached them on.

> **TIP**
> Over long distances, more parallel streams help: try `--streams 16`.

Seeing and removing paired devices

laptop

```
$ beam peers
Paired:
  desktop             cafb151299672aaa  192.168.1.7:7880  online

On this network:
  desktop             cafb151299672aaa  192.168.1.7:7880  via wlp1s0 (Wi-Fi)  [paired]
$ beam unpair desktop
forgot desktop
```

Unpairing only changes the machine you run it on: the laptop now refuses transfers from the desktop and can no longer send to it, but the desktop still lists the laptop until you run `beam unpair laptop` there too.

5 Using the terminal

Everything beam can do is available as a `beam` sub-command. This chapter covers daily use; chapter 12 lists every option.

Sending

```
$ beam send <file or folder>... --to <peer> [options]
```

- Send any mix of files and folders in one go. Folders are sent with everything inside them, including empty folders and symbolic links.
- `--to` takes a device name, a fingerprint prefix, or `host[:port]`.
- The progress bar shows percentage, amount, current speed, time left and — when compression is helping — how much smaller the data got.
- Press `Ctrl-C` to cancel. Running the same command later **resumes** where it stopped (chapter 8).

OPTION	MEANING
`--streams N`	Number of parallel connections (1–32). Default: the number of CPU cores, between 2 and 8.
`--compress MODE`	`auto` (default) compresses only data that benefits; `off` never compresses; `1`–`19` forces that zstd level (higher = smaller but slower).
`--code CODE`	Send to an unpaired machine using a one-shot code it gave you.
`--detach`	Start the transfer in the background and return immediately.

Receiving

On a paired machine there is nothing to do: transfers are accepted automatically and saved into the download folder. Each top-level file or folder you send becomes an entry there:

- Sending `~/Videos/trip` creates `~/Downloads/beam/trip/…`
- If something with that name already exists, beam never overwrites it — it picks a new name like `trip (1)` or `notes (1).txt`.
- While a file is arriving it is called `name.beam-part`; it gets its real name, permissions and timestamp once complete.

Asking before accepting

If you'd rather approve each incoming transfer, turn off auto-accept on the receiving machine:

desktop

```
$ beam config auto_accept off
$ beam ls
  id  started         peer            what              size          speed       state
   1  10-25 19:17  ← laptop          holiday      400.0 MiB   0.0%  0 kbit/s    waiting
$ beam accept 1        # or: beam reject 1
✓ received from laptop (400.0 MiB, 1 files) in 1.4s — 8.99 Gbit/s avg
  saved to /home/you/Downloads/beam/holiday
```

`beam accept` shows the progress bar until the transfer finishes. An unanswered request is declined after 10 minutes. The dashboard shows *Accept* / *Decline* buttons for waiting transfers.

Following transfers

laptop

```
$ beam send ~/Videos --to desktop --detach
transfer 5 started — follow it with `beam watch 5`
$ beam ls
  id  started         peer            what                size          speed
state
   5  10-25 19:07  → desktop         Videos            1.1 GiB   5.8%  207.7 Mbit/s
active
   4  10-25 19:07  → desktop         link benchmark        0 B   0.0%  24.20 Gbit/s   done
   3  10-25 19:07  → desktop         server.log       300.0 MiB 100.0%  8.43 Gbit/s    done
$ beam watch 5          # live progress bar; Ctrl-C here cancels the transfer
$ beam cancel 5         # stop it without watching
$ beam history          # the last 200 transfers, kept across restarts
```

Arrows show direction: `→` sent, `←` received.

Checking status and settings

laptop

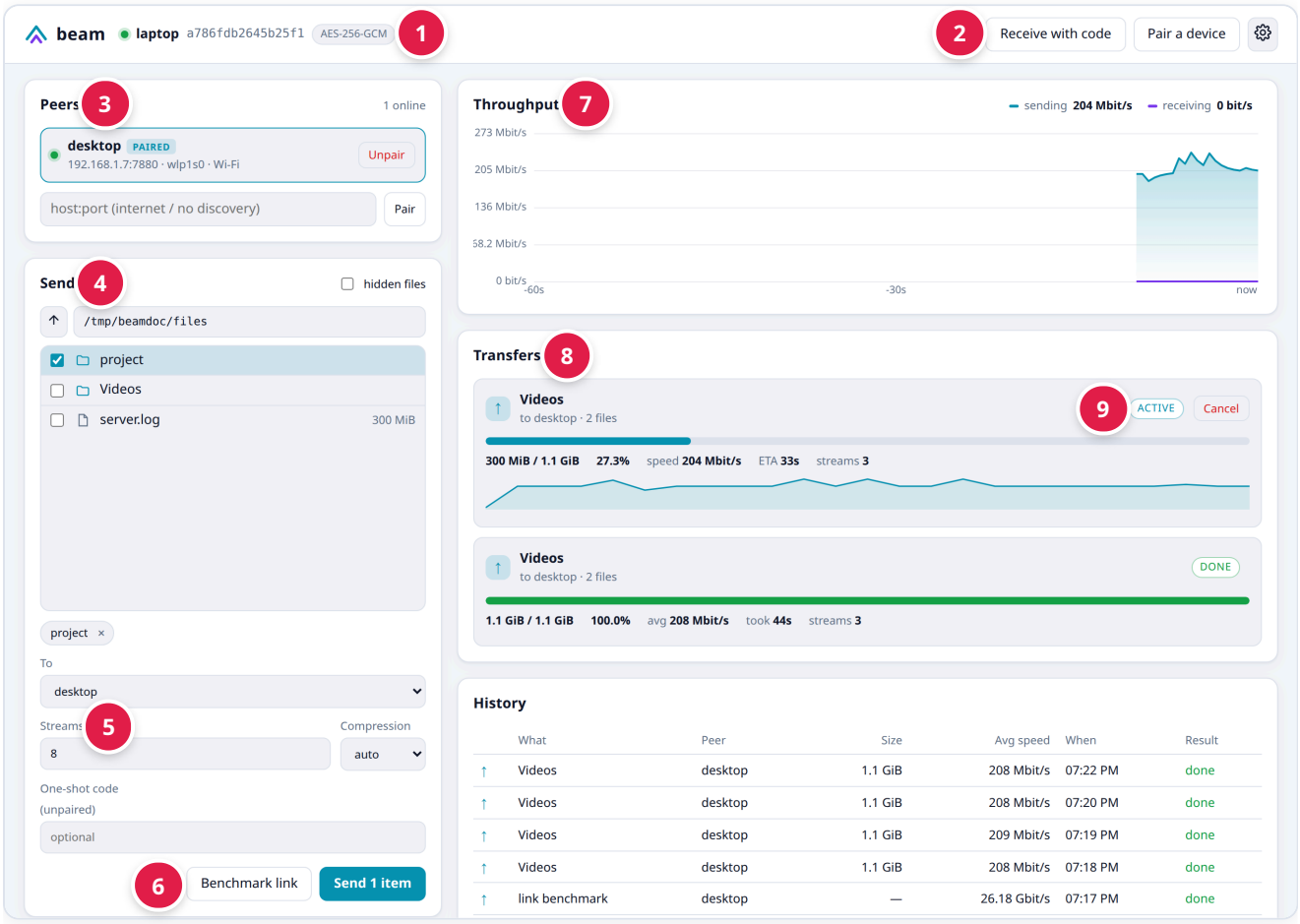
```
$ beam status
beam 0.1.0 — daemon running
  device       laptop (a786fdb2645b25f1)
  peer port    7878 (TCP), discovery UDP 7879
  web UI       http://127.0.0.1:8080  (open with `beam ui`)
  downloads    /home/you/Downloads/beam
  auto-accept  on from paired peers
  streams      8    compression auto    cipher AES-256-GCM
  interface    wlp1s0     192.168.1.7     Wi-Fi
$ beam config name "work-laptop"
$ beam config download_dir ~/Inbox
$ beam config compress off
$ beam stop             # stop the daemon (it restarts on the next beam command)
```

6 Using the web dashboard

The dashboard does everything the terminal does, with a file picker and live visuals. It runs inside the daemon — there is no separate server to install.

Opening it

Run `beam ui`. It opens your browser on `http://127.0.0.1:8080` with a secret access token attached to the address. The token keeps other websites you visit from controlling beam, so open the dashboard with `beam ui` rather than typing the address. If you see “Open beam from your terminal”, that is why. The dashboard is only reachable from the same computer.



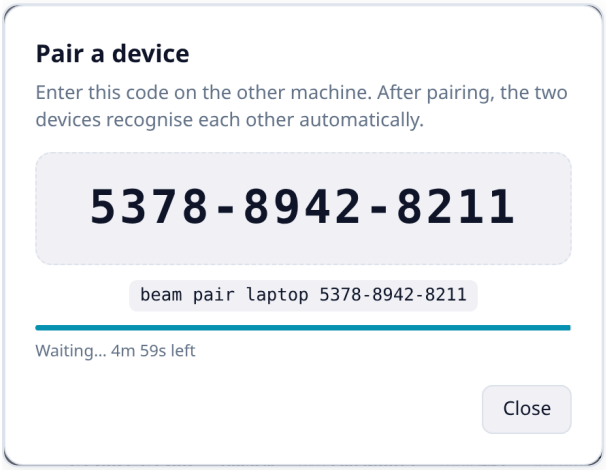
The dashboard during a transfer. Numbers refer to the list below.

1. **This device** — its name, fingerprint and the cipher in use. The dot is green while the page is connected to the daemon.
2. **Receive with code** and **Pair a device** show a code for the other machine; the gear opens Settings.
3. **Peers** — paired devices and new devices on the network, with address, interface and link speed. Click a paired device to make it the target; click *Pair* on a new one to enter its code. The box below pairs by address (internet).
4. **File picker** — type a folder path or use ↑. Click rows to select files and folders (double-click a folder to open it). Selections show as chips below.

5. **Options** — target device, number of streams, compression, and a one-shot code when sending to an unpaired device.
6. **Send** starts the transfer; **Benchmark link** runs a 10-second speed test to the target (chapter 7).
7. **Throughput** — total sending (cyan) and receiving (violet) speed over the last minute.
8. **Transfers** — one card per transfer with progress, speed, time left, compression savings and its own speed graph. Grey on the bar means data already present from an earlier attempt.
9. **State and actions** — Cancel, or Accept/Decline for incoming transfers waiting for approval. Errors appear in red under the card.

Below the transfers, **History** lists past transfers with size, average speed, time and result. Notifications pop up in the bottom-right corner when a transfer finishes or fails.

Pairing from the browser

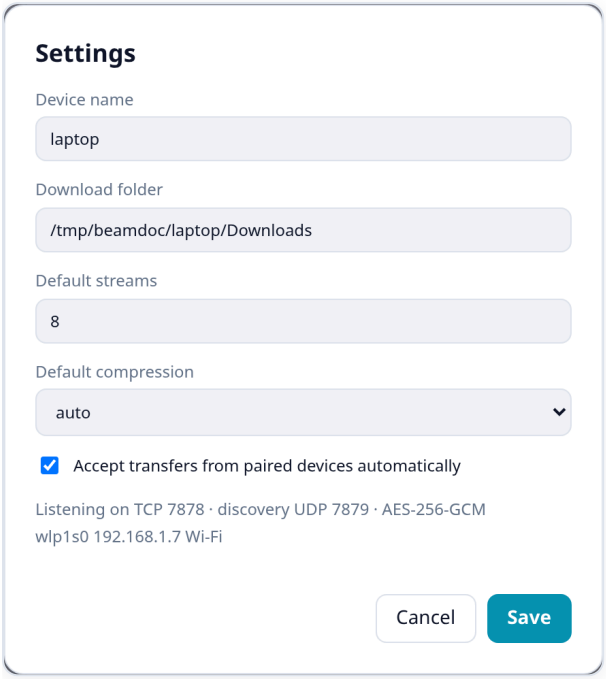


Pair a device. The code, the matching command for the other machine, and a countdown. The dialog closes by itself once pairing succeeds.

On the machine that *shows* the code, press **Pair a device**. On the other machine either run the displayed `beam pair …` command or click **Pair** next to the device in the Peers list and type the code.

Receive with code works the same way for one-shot transfers: give the code to the sender, who enters it in the *One-shot code* field (5) or uses `--code`.

Settings

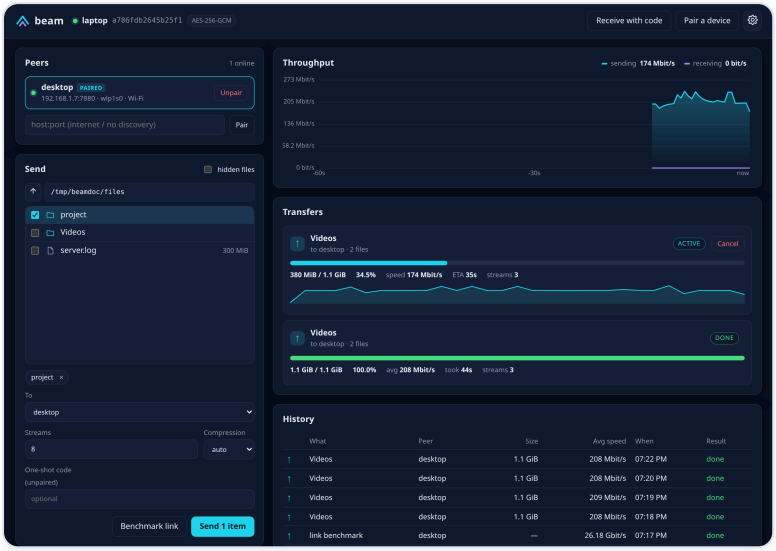


Settings. Stored in `~/.config/beam/config.json`; the same as `beam config`.

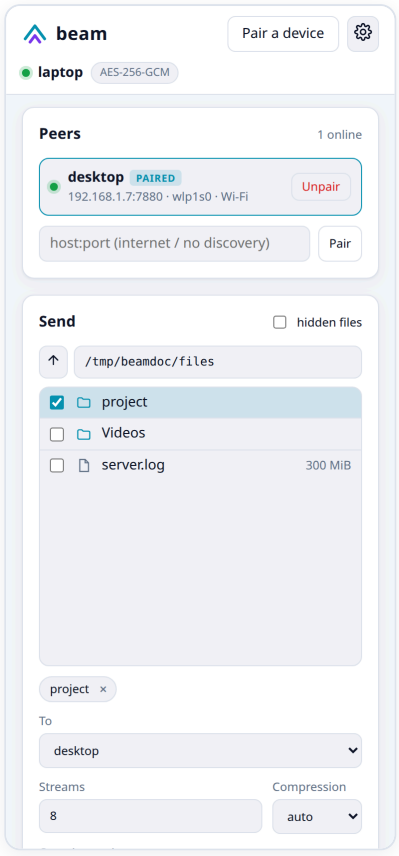
- **Device name** — how other machines see this one.
- **Download folder** — where received files go (an absolute path; created if missing).
- **Default streams** and **compression** — used when a send doesn't specify them.
- **Accept automatically** — turn off to approve each incoming transfer.

The bottom of the dialog lists the ports and the network interfaces with their link speed.

Dark mode and phones



Dark theme, chosen automatically from your system setting.



Narrow windows stack the panels in one column.

7 Getting the most speed

A transfer can never be faster than the slowest of three things: the network link, the disks at either end, and the CPU doing encryption and compression. beam is designed so the network or the disk is the limit.

Measure your link first

`beam bench` sends encrypted data from memory to memory for a few seconds — no disks involved — and reports what the connection plus encryption can do:

laptop

```
$ beam bench desktop --secs 10
27.5 GiB  27.99 Gbit/s  0:01
link to desktop: 26.18 Gbit/s  (32.4 GiB in 10.6s over 8 streams)
```

This example ran with both daemons on one computer, so no real network was involved; between two machines the result is limited by the link (about 0.94 Gbit/s on gigabit Ethernet).

Compare the benchmark with a real transfer. If transfers are much slower than the benchmark, the disk (or many tiny files) is the bottleneck, not the network.

What to expect

CONNECTION	REALISTIC CEILING	1 GB TAKES ABOUT
Wi-Fi 5 / Wi-Fi 6	200 – 900 Mbit/s, varies a lot	10 – 40 s
Gigabit Ethernet	~940 Mbit/s	~9 s
2.5 GbE	~2.35 Gbit/s	~3.5 s
10 GbE	~9.4 Gbit/s (fast NVMe needed)	~1 s
USB4 / Thunderbolt networking	10 – 30 Gbit/s	under 1 s

For reference, on a single laptop (loopback, RAM disk) beam measured about 13 Gbit/s for a 1 GiB file, 12 Gbit/s for a 20 000-file folder, and 22–25 Gbit/s in `beam bench`, with AES-256-GCM encryption on.

Use the fastest cable you have

- **Prefer Ethernet to Wi-Fi.** When a peer is reachable over several interfaces, beam automatically uses the one with the highest link speed — `beam peers` shows which one.
- **Direct cable.** Two machines connected by one Ethernet cable work fine; give each port an address (most desktops do this automatically with link-local addresses).

- **USB4 / Thunderbolt.** Connecting two machines with a Thunderbolt cable creates a `thunderbolt0` network interface. beam uses it like any other network — this is the fastest option for laptops.
- **Bluetooth is not supported**: at about 2 Mbit/s in practice it is hundreds of times slower than Ethernet.

Tuning knobs

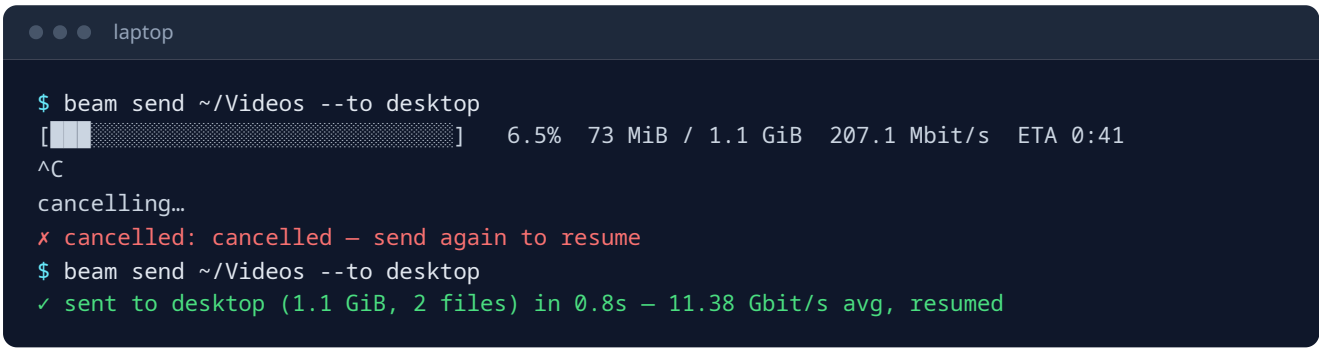
SITUATION	TRY
Internet or long-distance link	`--streams 16` (or up to 32) to keep more data in flight.
Slow link, compressible data (logs, text, databases, VM images)	`--compress 3` to `9`: smaller on the wire. `auto` already uses a fast level.
Very fast link, data known to be incompressible	`--compress off` saves a little CPU (auto mode already skips incompressible chunks after a quick test).
Old CPU without AES-NI	Nothing to do: beam switches to ChaCha20-Poly1305 automatically, which is fast in software.
Hundreds of thousands of small files	Already handled: files under 1 MiB are bundled into 4 MiB chunks. The receiving disk's speed at creating files becomes the limit.

8 Reliability and resuming

Large transfers get interrupted: laptops sleep, Wi-Fi drops, someone presses Ctrl-C. beam is built so that starting again costs only the part that didn't arrive.

Resuming

To resume, **run the same send again** — same files, same destination device. beam recognises it and transfers only the missing chunks:


laptop
$ beam send ~/Videos --to desktop
[███▒▒▒

When something goes wrong

Errors are reported on both sides in plain language, for example:

```
✗ failed: receiver: cannot write /home/you/Downloads/beam/Videos/day1.mp4: No space left on device
```

Fix the cause (here: free some space) and run the same send again — it continues where it stopped.

9 Security

This chapter explains what protects your data, for readers who want to know. You don't need it to use beam.

Device identity

On first start every machine creates a random key pair (X25519), stored in `~/.config/beam/identity.key` and readable only by you. Its fingerprint is what you see next to the device name. Pairing exchanges these public keys; afterwards each machine only accepts transfer sessions from keys in its paired list.

Pairing codes cannot be cracked offline

Pairing uses **CPace**, a password-authenticated key exchange (PAKE) over the ristretto255 group. Both machines turn the code into a mathematical starting point and exchange values derived from it. If both used the same code they arrive at the same secret key; if not, they don't — and the exchanged values reveal nothing that would let an eavesdropper test guesses later. An attacker gets exactly one online guess per code, and a failed guess burns the code.

Every transfer gets fresh keys

Each transfer between paired machines starts with a handshake modelled on the Noise “KK” pattern: both sides combine their permanent keys with brand-new temporary keys. Only the two genuine devices can compute the result, and recorded traffic stays unreadable even if a device key is stolen later (forward secrecy). Each parallel data connection derives its own keys, salted with a random value from the receiver, so recorded connections can't be replayed.

Encryption

All traffic after the handshake is encrypted and authenticated with **AES-256-GCM** (on CPUs with AES instructions) or **ChaCha20-Poly1305**, using libsodium. File names and folder structure are encrypted too — someone watching the network sees only that two machines are talking, and roughly how much data moves. Separately, on your local network each daemon announces its device name and fingerprint every two seconds so others can find it; choose a device name you don't mind neighbours on the same network seeing.

The receiving machine is protected

- Files can only be written inside the download folder: paths that are absolute, contain `..`, or would pass through a symbolic link created by the same transfer are rejected.
- Existing files are never overwritten.
- Unpaired machines are refused unless you armed a one-shot code yourself.

The dashboard

The web server listens on `127.0.0.1` only. Every request must carry the token from `~/.config/beam/web_token`, and requests whose `Host` or `Origin` isn't the local dashboard are rejected — this blocks malicious websites from using your browser to control beam (CSRF and DNS rebinding). The terminal talks to the daemon over a Unix socket that only your user can open.

> **KEEP IN MIND**
>
> Anyone who can log in as your user can use beam as you, like any other program. Treat `~/.config/beam` as private, and unpair devices you no longer own.

10 Configuration reference

Settings

Change settings with `beam config <key> <value>`, in the dashboard's Settings dialog, or by editing `~/.config/beam/config.json` (restart the daemon afterwards with `beam stop`).

KEY	DEFAULT	MEANING
`name`	host name	Device name shown to other machines.
`download_dir`	`~/Downloads/beam`	Where received files are saved.
`auto_accept`	`on`	Accept transfers from paired devices without asking.
`streams`	CPU cores, 2–8	Default number of parallel connections.
`compress`	`auto`	`auto`, `off`, or a zstd level `1`–`19`.
`peer_port`	`7878`	TCP port for transfers (config file or `beam daemon --port`).
`web_port`	`8080`	Dashboard port on 127.0.0.1 (config file or `beam daemon --web`; `0` disables it).

Files

PATH	CONTENTS
`~/.config/beam/config.json`	The settings above.
`~/.config/beam/identity.key`	This device's private key. Deleting it gives the device a new identity — all pairings must then be redone.
`~/.config/beam/peers.json`	Paired devices: name, public key, last known address.
`~/.config/beam/web_token`	Access token for the dashboard.
`~/.config/beam/history.json`	The last 200 transfers.
`~/.config/beam/daemon.log`	Daemon log when it was started automatically (with systemd: `journalctl --user -u beam`).
`<download_dir>/.beam/`	Progress journals of unfinished incoming transfers.
`$XDG_RUNTIME_DIR/beam.sock`	Socket the `beam` command uses to talk to the daemon.

Network ports

PORT	USED FOR	NEEDS TO BE OPEN?
`7878/tcp`	Pairing and transfers	Yes, on machines that receive (forward it on the router for internet use).
`7879/udp`	Discovery beacons (broadcast and multicast group 239.255.77.78)	Yes, for automatic discovery on the LAN.
`8080/tcp`	Dashboard, on 127.0.0.1 only	No — never reachable from the network.

Running several independent instances

`beam --home DIR …` (or the environment variable `BEAM_HOME=DIR`) uses a different configuration folder — and therefore a separate daemon, identity and pairing list. Give each instance its own ports:

```
$ beam --home ~/beam-test daemon --port 7880 --web 8081 --name test-instance
```

This is how the automated tests run two "machines" on one computer.

11 Troubleshooting and FAQ

Error messages

MESSAGE	WHAT IT MEANS AND WHAT TO DO
nothing discovered yet — is beam running on the other machine, and UDP 7879 open?	Start beam on the other machine (`beam status`) and open UDP 7879. Some Wi-Fi networks (guest networks, “client isolation”) block discovery — use the address instead: `--to 192.168.1.20`.
X is not paired — run `beam pair` on it, or use a one-shot code	You found the device but haven't paired with it yet (chapter 4).
… does not know this device — pair first with `beam pair`	The other machine doesn't (or no longer) have you in its paired list — e.g. it was reinstalled or unpaired you. Run `beam unpair` on your side and pair again.
… has no active code — run `beam pair` there first	There is no code waiting on the other machine, it expired, or it was already used. Generate a new one.
wrong code (the code is now used up)	The code didn't match. Run `beam pair` (or `recv --code`) again for a fresh code.
cannot connect to HOST:7878	The other daemon isn't running, the address is wrong, or a firewall blocks TCP 7878.
this transfer is already running	An identical transfer is still in progress. Wait for it, or cancel it first.
connection lost — run the same send again to resume	The network dropped or the other side stopped. Just repeat the command.
receiver: cannot write …: No space left on device	The receiving disk is full (or the folder isn't writable). Fix it, then resend to resume.
cannot listen on port 7878: Address already in use	Another program (or another beam) uses the port. Choose another with `beam daemon --port`.
“Open beam from your terminal” in the browser	The page was opened without the access token. Run `beam ui`.

Frequently asked questions

Where did my files go?

Into the receiver's download folder — `~/Downloads/beam` by default. The final line of `beam recv` / `beam accept` prints the exact path, and the dashboard shows it on the transfer card.

Can I send to several machines at once, or in both directions?

Yes. Start as many sends as you like (use `--detach` or several terminals); every daemon sends and receives at the same time.

Is it safe to use over the internet?

Yes — everything is encrypted and authenticated, and an open port 7878 only accepts paired devices (or an armed one-shot code). As with any exposed service, keep beam updated.

Why is my transfer slower than the benchmark?

The disk is usually the limit: spinning hard drives manage 100–250 MB/s, and creating very many tiny files is slow on any disk. Wi-Fi speed also varies from second to second — look at the throughput graph.

Does beam sync folders like rsync?

No. It copies what you send. Sending a folder again creates a new copy (`name (1)`) unless the previous transfer of the same, unchanged files was unfinished — then it resumes.

How do I see what the daemon is doing?

Read `~/.config/beam/daemon.log`, or run it in the foreground with extra detail: `beam stop` then `beam daemon -v`.

I reinstalled my OS. Do I need to pair again?

Yes, unless you kept `~/.config/beam`. A fresh install creates a new identity; unpair the old entry on the other machines and pair again.

12 Command reference

`beam [--home DIR] <command> [options]` — `<peer>` is a device name, a fingerprint prefix, or `host[:port]`.

COMMAND	DESCRIPTION
`send <paths>… --to <peer>`	Send files and folders. Options: `--streams N`, `--compress auto|off|1-19`, `--code CODE`, `--detach`. Ctrl-C cancels (resumable).
`recv --code`	Show a one-shot code (10 min, single use) and wait for the incoming transfer.
`pair`	Show a pairing code (5 min, one attempt) and wait until the other machine uses it.
`pair <peer> <code>`	Pair with the machine that shows the code.
`unpair <peer>`	Forget a paired device.
`peers`	Paired devices (online or not) and devices discovered on the network, with interface and link speed.
`ls` / `transfers`	Current and recent transfers with their ids.
`history`	The last 200 transfers, kept across restarts.
`watch <id>`	Show the live progress bar of a transfer. Ctrl-C cancels it.
`accept <id>`	Accept a waiting incoming transfer and follow it.
`reject <id>`	Decline a waiting incoming transfer.
`cancel <id>`	Stop a transfer (either direction). It can be resumed later.
`bench <peer> [--secs N] [--streams N]`	Measure link + encryption throughput without disk I/O (default 10 s, max 120 s).
`ui`	Open the dashboard in the browser (prints the address too).
`status` / `info`	Device, ports, download folder, defaults, cipher and network interfaces.
`config <key> <value>`	Change `name`, `download_dir`, `auto_accept` (on/off), `streams` or `compress`.
`daemon [options]`	Run the daemon in the foreground: `--port N`, `--web N`, `--dir DIR`, `--name NAME`, `-v` for debug logs. Options given here are saved.

COMMAND	DESCRIPTION
`stop`	Stop the daemon.
`--version`, `--help`	Print the version or the usage summary.

Exit status

`0` on success, `1` when a transfer or command fails, `2` for usage errors — useful in scripts:

```
$ beam send /srv/backups/nightly.tar.zst --to nas && echo "backup copied"
```

13 For developers

Project layout

PATH	CONTENTS
`src/main.c`	Command-line interface
`src/daemon.c`, `http.c`, `api.c`	Daemon loop and Unix-socket API, HTTP/SSE server, commands shared by CLI and dashboard
`src/peer.c`, `crypt.c`	Connection hellos, CPace pairing, KK handshake, encrypted channels
`src/sender.c`, `receiver.c`	The transfer engine
`src/manifest.c`, `journal.c`, `compress.c`	File lists and chunk layout, resume journal, adaptive zstd
`src/discovery.c`, `transfer.c`, `config.c`	LAN beacons, transfer state and history, settings and peer store
`src/proto.h`	Wire protocol definition
`web/`	Dashboard (plain HTML, CSS, JavaScript — no build step), embedded into the binary
`tests/`	Unit tests and the end-to-end script
`docs/`	This guide: source in `docs/guide/`, screenshots in `docs/img/`, tools in `docs/tools/`

Tests

terminal

```
$ make test
== build/test_crypto
  ok
== build/test_storage
  ok
all unit tests passed
$ make e2e
== pairing
  ✓ wrong pairing code rejected
  ✓ paired alpha ↔ bravo
== transfers
  ✓ 300 MiB file
  ✓ folder: 3000 small files, modes, mtimes, symlink, empty file/dir
  ✓ name clash saved as 'big (1).bin'
  ✓ unpaired device refused
  ✓ one-shot code transfer from an unpaired device
== resume
  ✓ killed sender mid-transfer, resumed without resending finished chunks
== web
  ✓ web API requires the token and a loopback Host
== bench
  ✓ link benchmark
all 10 end-to-end checks passed
```

Rebuilding this guide

The guide is an HTML page printed to PDF by headless Chrome (Node.js 22+ and Google Chrome or Chromium required):

```
$ make docs                # → docs/beam-user-guide.pdf
# refresh the dashboard screenshots (needs a running daemon, ideally mid-transfer):
$ node docs/tools/screenshots.mjs "$(beam ui | head -1)" docs/img ~/some/folder
```